We can see that the child class `VwGolf` has inherited the static `$noise` property and public `beepHorn()` method from the parent `Car` class. `VwGolf` has also implemented the abstract `makeNoise()` method (as it must do).

Abstract classes enable the sharing of code between child classes, in addition to controlling the interfaces of children. There is also another PHP mechanism for reusing code in classes that sits outside the inheritance hierarchy: traits, which we will look at later in the chapter.

Accessing Parent Members

It is sometimes necessary to access the parent implementation of an overloaded method, often during the execution of `__construct`. PHP provides the `parent` keyword to enable this. Suppose we had a parent class that was a general representation of something. If our software needed to model cars and motorbikes, we might have a parent `Vehicle` class. In this scenario, let's say we set the number of passengers on instantiation using the constructor method. Now let's imagine that we need to model another type of vehicle, which needs a more complex operation for initializing its `maxPassengers` property:

```
class Vehicle
{
    public function __construct(
        protected int $maxPassengers
    ) {
    }
}

class Train extends Vehicle
{
    public function __construct(int $carriages)
    {
        parent::__construct($carriages * 30);
    }
```